Основы SQL и CRUD-операции

Читаем, создаём, изменяем и удаляем данные PostgreSQL из Node.js

Результаты занятия

- ✓ Объяснять назначение SQL в реляционной базе данных
- ✓ Связывать CRUD-операции с командами SELECT, INSERT, UPDATE и DELETE
- ✓ Использовать WHERE для выбора нужных строк
- ✓ Сортировать результаты через ORDER BY
- ✓ Получать изменённые строки через RETURNING
- ✓ Выполнять параметризованные SQL-запросы через pool.query()
- ✓ Реализовывать полный CRUD в Product Service
- ✓ Сохранять разделение Route → Controller → Service → PostgreSQL

Учебные вопросы

1	**SQL и CRUD**	Назначение SQL и соответствие Create, Read, Update, Delete
2	**SELECT**	Чтение строк, WHERE, условия и ORDER BY
3	**INSERT**	Создание строк и получение результата через RETURNING
4	**UPDATE и DELETE**	Изменение и удаление выбранных строк
5	**CRUD в Node.js**	Реализация SQL-команд внутри productService.js

Учебный вопрос 1. SQL и CRUD

SQL становится основным способом работы Service с данными

Что такое SQL

- SQL — язык, используемый для работы со структурой и данными реляционных баз данных.
- SQL-команда отправляется серверу PostgreSQL.
- PostgreSQL выполняет команду над таблицами базы данных.
- Результатом может быть набор строк или информация об изменённых данных.
- SQL используется независимо от JavaScript и Express.
- Node.js передаёт SQL PostgreSQL через драйвер pg.

Какие задачи решает SQL

- Создание и изменение структуры таблиц.
- Добавление новых строк.
- Получение существующих данных.
- Изменение значений в строках.
- Удаление строк.
- Фильтрация, сортировка и объединение данных.

Главная идея

В этой лекции сосредоточимся на командах работы с данными: SELECT, INSERT, UPDATE и DELETE.

CRUD и SQL

CRUD	Назначение	SQL-команда	Product API
Create	Создать данные	INSERT	POST /api/products
Read	Получить данные	SELECT	GET /api/products
Update	Изменить данные	UPDATE	PATCH /api/products/:id
Delete	Удалить данные	DELETE	DELETE /api/products/:id

CRUD: от HTTP-метода к SQL-команде

HTTP / REST API

Клиент отправляет HTTP-запрос к API

POST /products
HTTP/1.1

POST → **Create**
Создать

GET /products
HTTP/1.1

GET → **Read**
Прочитать

PATCH /products/1
HTTP/1.1

PATCH → **Update**
Обновить

DELETE /products/1
HTTP/1.1

DELETE → **Delete**
Удалить

products

Один и тот же ресурс для всех операций CRUD

Service / PostgreSQL

Service выполняет SQL-команду в базе данных

INSERT →
```
INSERT INTO products
(...) VALUES (...);
```

SELECT →
```
SELECT * FROM products
WHERE ...;
```

UPDATE →
```
UPDATE products
SET ...
WHERE ...;
```

DELETE →
```
DELETE FROM products
WHERE ...;
```

PostgreSQL

Надёжная база для ваших данных

HTTP описывает взаимодействие клиента с API, а SQL — взаимодействие Service с PostgreSQL.

Общая форма SQL-команды

```
SELECT id, name, price
FROM products
WHERE price > 10000
ORDER BY price DESC;
```

Что важно

- SELECT определяет, какие данные получить.
- FROM указывает таблицу.
- WHERE ограничивает набор строк.
- ORDER BY задаёт порядок результата.
- Точка с запятой завершает SQL-команду.

Имена в SQL и JavaScript

```
SELECT in_stock AS "inStock"
FROM products;
```

Что важно

- В PostgreSQL часто используют snake_case для имён столбцов.
- В JavaScript часто используется camelCase.
- Например, столбец in_stock соответствует свойству inStock.
- SQL alias позволяет изменить имя поля в результате запроса.
- Для сохранения регистра camelCase alias можно заключить в двойные кавычки.

Учебный вопрос 2. SELECT

Получаем нужные строки и столбцы из таблицы products

SELECT читает данные

- SELECT получает данные из одной или нескольких таблиц и формирует результирующий набор строк.
- SELECT * получает все столбцы.
- Лучше явно указывать необходимые столбцы.
- FROM определяет источник данных.
- SELECT не изменяет строки таблицы.
- Результат запроса в Node.js доступен через result.rows.

SELECT * и явный список столбцов

SELECT *

- Получает все столбцы
- Удобен для быстрого просмотра
- Результат зависит от структуры таблицы
- Может передавать лишние данные

Явный список

- Получает только нужные поля
- Контролирует структуру результата
- Можно использовать alias
- Предпочтителен для REST API

Получаем все товары

```
SELECT
  id,
  name,
  price,
  category,
  in_stock AS "inStock"
FROM products
ORDER BY id;
```

Что важно

- Запрос возвращает все строки products.
- API получает только нужные столбцы.
- in_stock преобразуется в inStock.
- ORDER BY id сортирует товары по идентификатору.

WHERE фильтрует строки

- WHERE задаёт условие, которому должны соответствовать строки, участвующие в операции.
- WHERE используется с SELECT, UPDATE и DELETE.
- Можно сравнивать числа, строки и логические значения.
- Несколько условий объединяются через AND и OR.
- Без WHERE операция может затронуть все строки.
- Условие должно точно выражать, какие данные требуется выбрать.

Операторы условий

Оператор	Смысл	Пример
=	Равно	category = 'Accessories'
<>	Не равно	category <> 'Monitors'
>	Больше	price > 10000
<	Меньше	price < 50000
>=	Больше или равно	price >= 15000
<=	Меньше или равно	price <= 85000
AND	Оба условия истинны	price > 10000 AND in_stock = true
OR	Хотя бы одно условие истинно	category = 'Monitors' OR category = 'Laptops'

Фильтрация товаров

Запрос: Получить товары в наличии дороже 10000 ₸

```
SELECT id, name, price
FROM products
WHERE price > 10000
  AND in_stock = true
ORDER BY price ASC;
```

Результат

```
1 | Keyboard | 15000
4 | SSD 1TB | 45000
```

Пояснение

- WHERE оставляет только подходящие строки, а ORDER BY сортирует результат по цене.

Как SELECT превращает таблицу в нужный результат

Один запрос SQL последовательно выбирает таблицу, фильтрует строки, оставляет нужные столбцы и сортирует результат.

SQL выполняется по частям, и каждая часть сужает и преобразует данные.

Исходная таблица products

Все товары в базе данных

id	name	price	category	in_stock
1	Ноутбук	75000	Ноутбуки	true
2	Смартфон	30000	Смартфоны	true
3	Наушники	5000	Аудио	true
4	Монитор	15000	Мониторы	false
5	Клавиатура	7000	Аксессуары	true
6	Планшет	20000	Планшеты	true
7	Мышь	3000	Аксессуары	true
8	Сервер	120000	Серверы	true

Таблица **products** содержит все товары (8 строк, 5 столбцов).

1 FROM products

выбираем таблицу **products**

id	name	price	category	in_stock
1	Ноутбук	75000	Ноутбуки	true
2	Смартфон	30000	Смартфоны	true
3	Наушники	5000	Аудио	true
4	Монитор	15000	Мониторы	false
5	Клавиатура	7000	Аксессуары	true
6	Планшет	20000	Планшеты	true
7	Мышь	3000	Аксессуары	true
8	Сервер	120000	Серверы	true

На этом этапе просто берём таблицу **products**. Все строки и столбцы остаются без изменений.

2 WHERE price > 10000 AND in_stock = true

отбрасываем неподходящие строки

id	name	price	category	in_stock
1	Ноутбук	75000	Ноутбуки	true
2	Смартфон	30000	Смартфоны	true
3	Наушники	5000	Аудио	true ✗
4	~~Монитор~~	~~15000~~	~~Мониторы~~	~~false~~ ✗
5	~~Клавиатура~~	~~7000~~	~~Аксессуары~~	~~true~~ ✗
6	Планшет	20000	Планшеты	true
7	Мышь	3000	Аксессуары	true ✗
8	Сервер	120000	Серверы	true

Оставляем только строки, где **price > 10000** и **in_stock = true**.

Осталось **4 строки**.

3 SELECT id, name, price

оставляем нужные столбцы

id	name	price	category	in_stock
1	Ноутбук	75000	....	....
2	Смартфон	30000	....	....
6	Планшет	20000	....	....
8	Сервер	120000	....	....

Оставляем только нужные столбцы: **id, name, price**.

Столбцы **category** и **in_stock** больше не участвуют в результате.

4 ORDER BY price ASC

сортируем результат по возрастанию цены

id	name	price
1	Ноутбук	75000
2	Смартфон	30000
6	Планшет	20000
8	Сервер	120000

↓

id	name	price
6	Планшет	20000
2	Смартфон	30000
1	Ноутбук	75000
8	Сервер	120000

Сортируем строки по столбцу **price** в порядке возрастания (ASC — по умолчанию).

Итоговый набор строк

Результат выполнения запроса

id	name	price
6	Планшет	20000
2	Смартфон	30000
1	Ноутбук	75000
8	Сервер	120000

Готово!
SELECT вернул нужные данные из таблицы products: 4 строки, 3 столбца, отсортированные по цене.

FROM выбирает таблицу, **WHERE** фильтрует строки, **SELECT** оставляет нужные столбцы, **ORDER BY** сортирует результат.

SQL — это последовательная обработка данных.
Каждая часть запроса приближает нас к нужному результату.

SELECT по id из Node.js

```
const result = await pool.query(
  `SELECT
     id,
     name,
     price,
     category,
     in_stock AS "inStock"
   FROM products
   WHERE id = $1`,
  [id]
);

return result.rows[0];
```

Что важно

- $1 является параметром запроса.
- id передаётся отдельно в массиве значений.
- result.rows[0] содержит найденный товар.
- Если подходящей строки нет, значение будет undefined.

Учебный вопрос 3. INSERT

Добавляем новые строки и получаем созданный объект

INSERT создаёт новую строку

- INSERT INTO указывает таблицу назначения.
- После имени таблицы перечисляются заполняемые столбцы.
- VALUES содержит значения новой строки.
- id можно не передавать, если он создаётся PostgreSQL автоматически.
- Ограничения таблицы проверяются при вставке.

Добавляем товар вручную

```
INSERT INTO products
  (name, price, category, in_stock)
VALUES
  ('SSD 2TB', 65000, 'Components', true);
```

Что важно

- INSERT создаёт одну новую строку.
- id отсутствует в списке столбцов.
- PostgreSQL создаёт id автоматически.
- Типы и ограничения значений проверяются таблицей.

RETURNING возвращает изменённую строку

```
INSERT INTO products
  (name, price, category, in_stock)
VALUES
  ('SSD 2TB', 65000, 'Components', true)
RETURNING id, name, price, category,
          in_stock AS "inStock";
```

Что важно

- INSERT изменяет данные, но приложению часто нужен созданный объект.
- RETURNING позволяет получить значения вставленной строки.
- Можно вернуть автоматически созданный id.
- Можно вернуть все поля, необходимые REST API.
- Дополнительный SELECT после INSERT не требуется.

Слайд №24. POST /api/products → INSERT → RETURNING

Клиент отправляет JSON → Controller передаёт данные в Service → параметризованный **INSERT** создаёт строку в PostgreSQL → **RETURNING** возвращает созданный объект.

1 HTTP-клиент

Отправляет запрос с JSON



POST /api/products

Тело запроса (JSON)

```
{
  "name": "Gaming Mouse",
  "price": 18000,
  "category": "Accessories",
  "inStock": true
}
```

Клиент отправляет HTTP POST с данными нового товара.

2 Controller

Получает данные из req.body

productController.js

```
export async function
createProduct(req, res, next) {
  try {
    const product =
      await addProduct(req.body);
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

req.body = { ... }

```
{
  name: "Gaming Mouse",
  price: 18000,
  category: "Accessories",
  inStock: true
}
```

Controller не содержит SQL. Он получает данные, вызывает Service и отправляет ответ клиенту.

3 Product Service

Выполняет SQL-запрос через pool.query()

productService.js

```
export async function
addProduct(data) {
 const result = await pool.query(
  `INSERT INTO products
    (name, price, category, in_stock)
    VALUES ($1, $2, $3, $4)
    RETURNING
      id, name, price, category,
      in_stock AS "inStock"`,
   [data.name, data.price,
    data.category, data.inStock]
  );
 return result.rows[0];
}
```

Service содержит SQL.
Он использует параметризованный запрос и возвращает созданную строку (result.rows[0]).

4 PostgreSQL

Создаёт новую строку и назначает id



PostgreSQL

Таблица products (после INSERT)

id	name	price	category	in_stock
1	Laptop	42000	Laptops	true
2	Monitor	85000	Monitors	true
3	Keyboard	15000	Accessories	true
4	Gaming Mouse	18000	Accessories	true

PostgreSQL автоматически создаёт id (GENERATED AS IDENTITY) и сохраняет новую строку в таблице.

5 RETURNING

Возвращает созданную строку

Результат SQL (result.rows[0])

```
{
  "id": 4,
  "name": "Gaming Mouse",
  "price": 18000,
  "category": "Accessories",
  "inStock": true
}
```

RETURNING позволяет получить созданную строку сразу, без дополнительного SELECT.

6 Controller

Формирует HTTP-ответ

Controller получает созданный объект и отправляет его клиенту.

```
res.status(201)
  .json(product);
```

7 Ответ клиенту

201 Created + JSON

201 Created

```
{
  "id": 4,
  "name": "Gaming Mouse",
  "price": 18000,
  "category": "Accessories",
  "inStock": true
}
```

Клиент получает JSON нового товара со статусом 201.

Ключевые моменты

- POST создаёт новый ресурс.
- Данные передаются в JSON (req.body).
- Service выполняет параметризованный INSERT.
- RETURNING возвращает созданную строку.
- Controller отправляет ответ со статусом 201.

Преимущества RETURNING

- Сразу получаем созданный объект (включая id).
- Не нужен дополнительный SELECT.
- Работает для INSERT, UPDATE и DELETE.
- Обеспечивает простую и надёжную реализацию API.

Важно помнить

- Используйте параметризованные запросы ($1, $2, ...).
- SQL находится только в Service.
- Controller и Router не содержат SQL.
- Статус 201 означает успешное создание ресурса.

Create в Product Service

```
export async function addProduct(data) {
  const result = await pool.query(
    `INSERT INTO products
       (name, price, category, in_stock)
     VALUES ($1, $2, $3, $4)
     RETURNING
       id,
       name,
       price,
       category,
       in_stock AS "inStock"`,
    [data.name, data.price, data.category, data.inStock]
  );

  return result.rows[0];
}
```

Что важно

- Данные товара передаются отдельно от SQL.
- $1, $2, $3 и $4 соответствуют элементам массива.
- PostgreSQL создаёт новую строку.
- RETURNING возвращает созданный товар.
- Controller может отправить этот объект клиенту со статусом 201.

От POST до созданной строки

Запрос: POST /api/products

```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true
}
```

```
{
  "id": 4,
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true
}
```

Пояснение

- Controller передаёт req.body в Service, Service выполняет INSERT ... RETURNING, а созданная строка становится JSON-ответом API.

Учебный вопрос 4. UPDATE и DELETE

Изменяем и удаляем только те строки, которые действительно выбраны

UPDATE изменяет существующие строки

- UPDATE указывает изменяемую таблицу.
- SET задаёт новые значения столбцов.
- WHERE определяет строки для изменения.
- RETURNING позволяет получить новое состояние строки.
- Без WHERE команда UPDATE может изменить все строки таблицы.

Изменяем цену товара

```
UPDATE products
SET price = 42000
WHERE id = 4
RETURNING id, name, price, category,
          in_stock AS "inStock";
```

Что важно

- SET изменяет значение price.
- WHERE id = 4 выбирает один товар.
- RETURNING возвращает строку после изменения.
- Если id отсутствует, RETURNING не вернёт строку.

PATCH требует частичного обновления

- Наш REST API использует PATCH /api/products/:id.
- Клиент может передать только одно или несколько изменяемых полей.
- Непереданные поля должны сохранить прежние значения.
- Для учебного проекта можно использовать COALESCE с параметрами.
- Validation middleware уже проверяет значения до Service.

Частичное UPDATE через COALESCE

```
UPDATE products
SET
  name = COALESCE($1, name),
  price = COALESCE($2, price),
  category = COALESCE($3, category),
  in_stock = COALESCE($4, in_stock)
WHERE id = $5
RETURNING id, name, price, category,
          in_stock AS "inStock";
```

Что важно

- Если параметр содержит новое значение, столбец изменяется.
- Если параметр равен NULL, сохраняется текущее значение столбца.
- $5 содержит id изменяемого товара.
- Так один фиксированный SQL-запрос поддерживает частичное обновление.

Update в Product Service

```
export async function updateProductById(id, changes) {
  const result = await pool.query(
    `UPDATE products
     SET
       name = COALESCE($1, name),
       price = COALESCE($2, price),
       category = COALESCE($3, category),
       in_stock = COALESCE($4, in_stock)
     WHERE id = $5
     RETURNING
       id, name, price, category,
       in_stock AS "inStock"`,
    [
      changes.name ?? null,
      changes.price ?? null,
      changes.category ?? null,
      changes.inStock ?? null,
      id
    ]
  );
  return result.rows[0];
}
```

Что важно

- Service получает id и объект changes.
- Отсутствующие свойства преобразуются в null.
- SQL-параметры передаются отдельно.
- RETURNING возвращает обновлённый товар.
- Если строка не найдена, result.rows[0] будет undefined.

DELETE удаляет строки

- DELETE FROM указывает таблицу.
- WHERE определяет удаляемые строки.
- RETURNING позволяет получить данные удалённой строки.
- После DELETE строка больше не существует в таблице.
- Без WHERE команда DELETE может удалить все строки.

Главная идея

UPDATE и DELETE требуют особенно внимательной проверки WHERE.

Удаляем один товар

```
DELETE FROM products
WHERE id = $1
RETURNING id, name, price, category,
          in_stock AS "inStock";
```

Что важно

- $1 содержит id удаляемого товара.
- Удаляется только строка, соответствующая WHERE.
- RETURNING возвращает удалённый объект.
- Если товар не найден, result.rows будет пустым.

Delete в Product Service

```
export async function deleteProductById(id) {
  const result = await pool.query(
    `DELETE FROM products
     WHERE id = $1
     RETURNING
       id,
       name,
       price,
       category,
       in_stock AS "inStock"`,
    [id]
  );

  return result.rows[0];
}
```

Что важно

- Controller передаёт обычное значение id.
- Service выполняет параметризованный DELETE.
- Удалённая строка возвращается через RETURNING.
- undefined позволяет Controller сформировать 404.

PostgreSQL
Надёжные данные начинаются с правильных запросов

Почему WHERE критичен для UPDATE и DELETE

WHERE определяет, какие строки будут затронуты. Без него запрос применяется ко всей таблице.

Маленькое условие — большая защита ваших данных!

БЕЗОПАСНЫЙ СЦЕНАРИЙ

Используем WHERE — изменяется или удаляется только нужная строка.

1 UPDATE с WHERE

```
UPDATE products
SET price = 42000
WHERE id = 15;
```

Таблица products (до обновления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
15	Клавиатура	**2500**	Аксессуары
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Выполняем запрос...

Таблица products (после обновления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
15	Клавиатура	**42000**	Аксессуары
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Затрагивается только одна строка
(id = 15). Изменилось только её значение.

2 DELETE с WHERE

```
DELETE FROM products
WHERE id = 15;
```

Таблица products (до удаления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
15	Клавиатура	**2500**	Аксессуары
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Выполняем запрос...

Таблица products (после удаления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Удаляется только одна строка
(id = 15). Остальные данные остаются.

ОПАСНЫЙ СЦЕНАРИЙ

Без WHERE запрос применяется ко всей таблице!

БЕЗ WHERE ОЧЕНЬ ОПАСНО!

1 UPDATE без WHERE

```
UPDATE products
SET price = 42000;
```

Без WHERE!

Таблица products (до обновления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
15	Клавиатура	2500	Аксессуары
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Выполняем запрос...

Таблица products (после обновления)

id	name	price	category
10	Ноутбук	**42000**	Электроника
11	Смартфон	**42000**	Электроника
15	Клавиатура	**42000**	Аксессуары
18	Монитор	**42000**	Электроника
20	Мышь	**42000**	Аксессуары

Изменяются все строки!
Цена у всех товаров стала 42000.

2 DELETE без WHERE

```
DELETE FROM products;
```

Без WHERE!

Таблица products (до удаления)

id	name	price	category
10	Ноутбук	55000	Электроника
11	Смартфон	38000	Электроника
15	Клавиатура	2500	Аксессуары
18	Монитор	24000	Электроника
20	Мышь	1500	Аксессуары

Выполняем запрос...

Таблица products (после удаления)

id	name	price	category

Таблица пуста
Все строки удалены!

Удаляются все строки!
Таблица оказывается пустой.

ГЛАВНЫЙ ВЫВОД:

WHERE фильтрует, какие строки будут затронуты.
Без **WHERE** команда UPDATE или DELETE применяется ко всей таблице.

Пишите запросы осознанно!

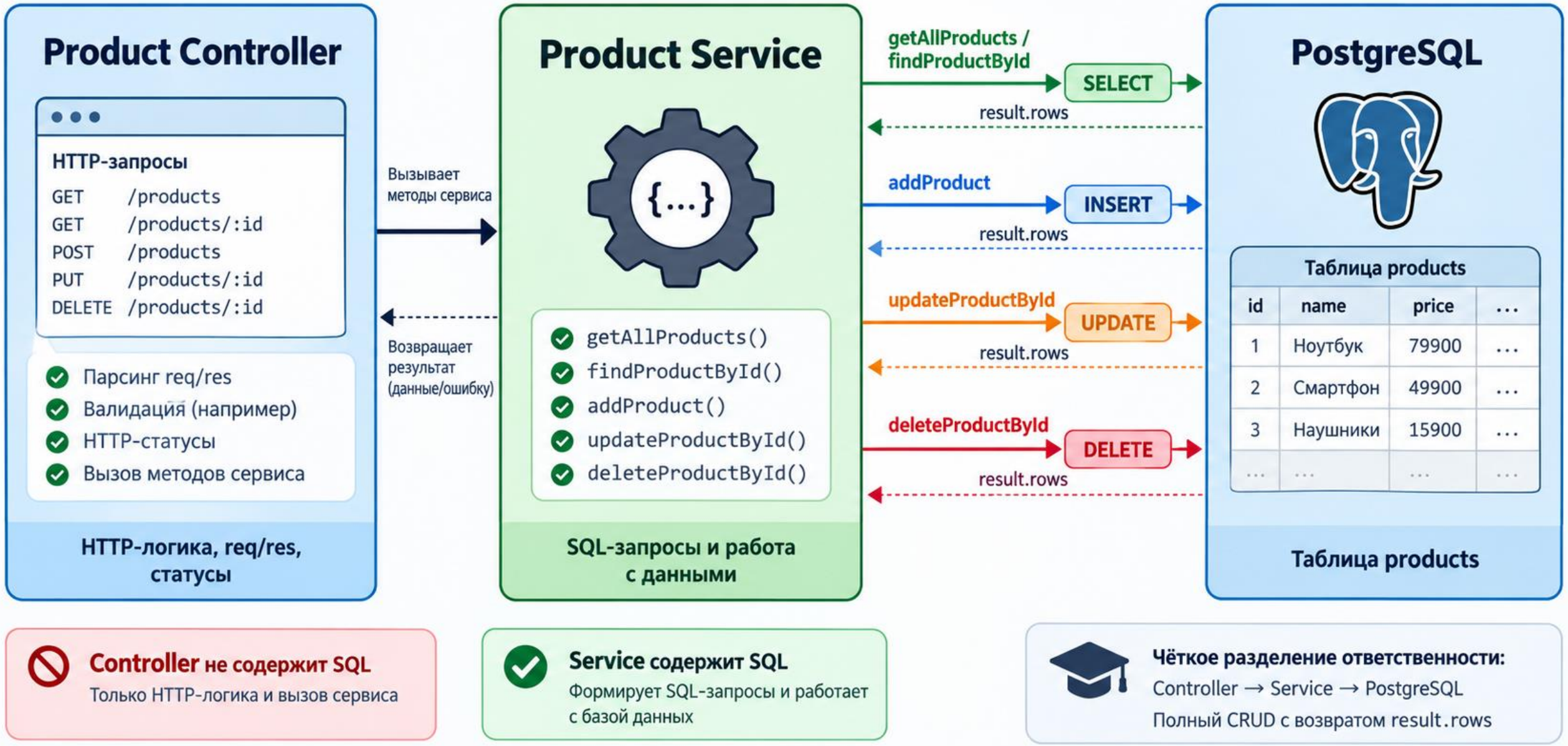
Учебный вопрос 5. CRUD в Node.js

Полностью заменяем операции JavaScript-массива SQL-командами

Как изменился Product Service

Операция	Раньше	Теперь
Read all	return products	SELECT ... FROM products
Read one	products.find()	SELECT ... WHERE id = $1
Create	products.push()	INSERT ... RETURNING
Update	Object.assign()	UPDATE ... WHERE ... RETURNING
Delete	products.splice()	DELETE ... WHERE ... RETURNING

Полный CRUD через Product Service и PostgreSQL



Controller не содержит SQL
Только HTTP-логика и вызов сервиса

Service содержит SQL
Формирует SQL-запросы и работает с базой данных

Чёткое разделение ответственности:
Controller → Service → PostgreSQL
Полный CRUD с возвратом result.rows

Объект результата node-postgres

- pool.query() возвращает объект результата.
- result.rows содержит строки, возвращённые SQL-командой.
- result.rows[0] удобно использовать для одной строки.
- result.rowCount показывает количество затронутых или возвращённых строк.
- SELECT и команды с RETURNING позволяют работать с rows одинаковым способом.

Controller создания почти не меняется

```
export async function createProduct(req, res, next) {
  try {
    const product = await addProduct(req.body);
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller по-прежнему получает данные из req.body.
- SQL находится только в Service.
- Controller ожидает асинхронную операцию через await.
- Созданный объект возвращается клиенту со статусом 201.
- Ошибки PostgreSQL передаются errorHandler.

Controller UPDATE использует результат SQL

```
export async function updateProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await updateProductById(id, req.body);

    if (!product) {
      return next(new AppError('Product not found', 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Controller не знает, как именно PostgreSQL выполняет UPDATE.
- Service возвращает обновлённую строку или undefined.
- Отсутствие строки преобразуется в HTTP 404.
- Публичный REST endpoint остаётся прежним.

Controller DELETE также не содержит SQL

```
export async function deleteProduct(req, res, next) {
  try {
    const id = Number(req.params.id);
    const product = await deleteProductById(id);

    if (!product) {
      return next(new AppError('Product not found', 404));
    }

    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- HTTP-логика остаётся в Controller.
- SQL DELETE выполняется в Service.
- Удалённая строка возвращается через RETURNING.
- Controller формирует JSON-ответ.

Один REST API — четыре SQL-команды

Один интерфейс. Одна архитектура. Четыре операции с данными.

HTTP endpoints

(клиент → сервер)

- POST /api/products
- GET /api/products/:id
- PATCH /api/products/:id
- DELETE /api/products/:id

Слои приложения (Backend)

Route

выбор endpoint

Controller

HTTP-логика, req/res

Service

SQL-запросы

Controller не содержит SQL

Service содержит SQL

PostgreSQL

(база данных)

products

id	name	price	stock	...
1	Ноутбук	99990	10	...
2	Мышь	1590	50	...
3	Клавиатура	2990	30	...
...	...	...	...	...

```
INSERT INTO products(...)
SELECT * FROM products WHERE id = ...
UPDATE products SET ... WHERE id = ...
DELETE FROM products WHERE id = ...
```

RETURNING / rows (результат запроса)

JSON Response (данные в формате JSON)

Ответ: данные возвращаются от базы через Service и Controller клиенту

Клиент

(браузер, мобильное приложение и т.д.)

Запрос (HTTP → SQL)

Ответ (SQL → JSON)

REST API — это один интерфейс, а CRUD — это всего лишь разные SQL-команды внутри.

Полный CRUD-цикл в PostgreSQL

1	**Create**	POST /api/products вызывает INSERT ... RETURNING и создаёт постоянную строку.
2	**Read**	GET /api/products/:id вызывает SELECT ... WHERE id = $1 и читает созданную строку.
3	**Update**	PATCH /api/products/:id вызывает UPDATE ... WHERE id = $5 RETURNING и изменяет выбранные поля.
4	**Read after Update**	Повторный SELECT подтверждает новое состояние данных.
5	**Delete**	DELETE /api/products/:id вызывает DELETE ... WHERE id = $1 RETURNING.
6	**Read after Delete**	Следующий SELECT не находит строку, поэтому API возвращает 404.

Правило параметризованных запросов

- Значения пользователя не вставляются в SQL через конкатенацию строк.
- В SQL используются параметры $1, $2, $3 и далее.
- Фактические значения передаются отдельным массивом.
- Это правило применяется к SELECT, INSERT, UPDATE и DELETE.
- Имена таблиц и столбцов нельзя подставлять обычными параметрами $1.

Главная идея

SQL-команда и пользовательские значения должны передаваться PostgreSQL раздельно.

Мини-практика: переведите CRUD Product API на SQL

Замените оставшиеся операции массива products[] командами PostgreSQL.

Задание

- Проверить SELECT всех товаров
- Реализовать SELECT по id
- Реализовать INSERT ... RETURNING
- Реализовать PATCH через UPDATE ... RETURNING
- Реализовать DELETE ... RETURNING
- Использовать параметры $1, $2 и далее
- Проверить POST, GET, PATCH и DELETE через HTTP-клиент
- Убедиться, что данные сохраняются после перезапуска Node.js

Критерии проверки

- В productService.js больше нет массива products[]
- Все CRUD-операции выполняются PostgreSQL
- POST возвращает созданную строку
- PATCH возвращает обновлённую строку
- DELETE возвращает удалённую строку
- Несуществующий id приводит к 404
- SQL отсутствует в Routes и Controllers

Контрольные вопросы

1. 1. Какие четыре SQL-команды соответствуют CRUD?
2. 2. Для чего используется WHERE?
3. 3. Чем SELECT * отличается от явного списка столбцов?
4. 4. Для чего используется ORDER BY?
5. 5. Что делает INSERT?
6. 6. Зачем PostgreSQL предоставляет RETURNING?
7. 7. Почему UPDATE и DELETE без WHERE опасны?
8. 8. Как реализовать частичное обновление товара?
9. 9. Что находится в result.rows после pool.query()?
10. 10. Почему пользовательские значения нужно передавать через $1, $2 и массив параметров?

Итоги занятия

1. SQL является основным языком работы приложения с PostgreSQL.
2. CRUD соответствует командам INSERT, SELECT, UPDATE и DELETE.
3. WHERE определяет строки, над которыми выполняется операция.
4. ORDER BY управляет порядком результатов SELECT.
5. RETURNING позволяет получить созданную, изменённую или удалённую строку без дополнительного SELECT.
6. Пользовательские значения передаются в SQL через параметры $1, $2 и далее.
7. Product Service полностью заменяет операции массива SQL-командами.
8. Routes и Controllers не содержат SQL и сохраняют прежние обязанности.
9. После перехода на PostgreSQL полный REST CRUD работает с постоянными данными.